

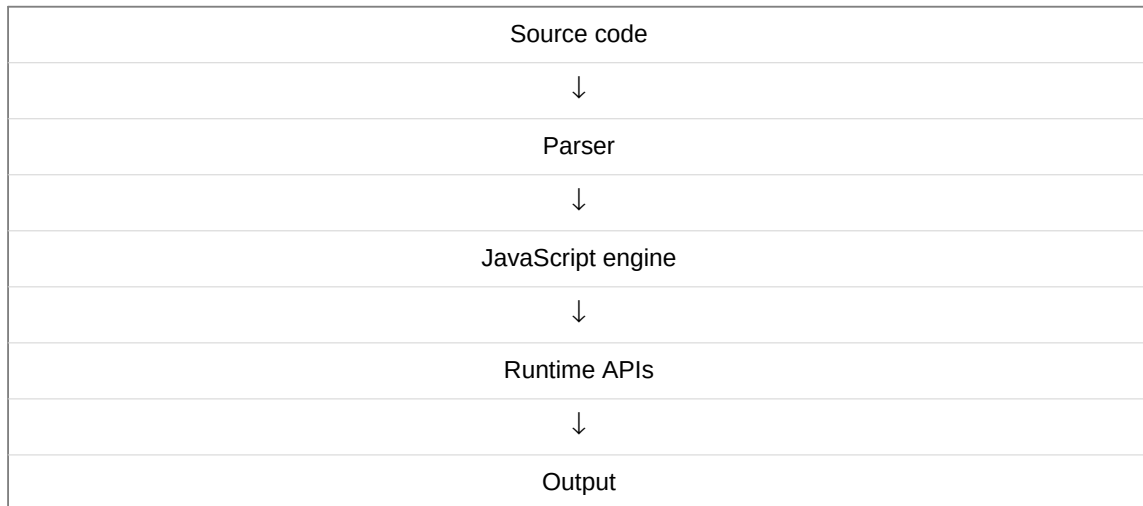
1. JavaScript & Runtime

Definition & Theory

JavaScript is a programming language used to create application logic. A runtime such as a browser, Node.js, or Bun provides the environment and additional APIs needed to execute JavaScript.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const message = "Hello JavaScript";  
console.log(message);
```

Conclusion

JavaScript code is executed by an engine inside a runtime; the runtime supplies capabilities such as timers, networking, or the DOM.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

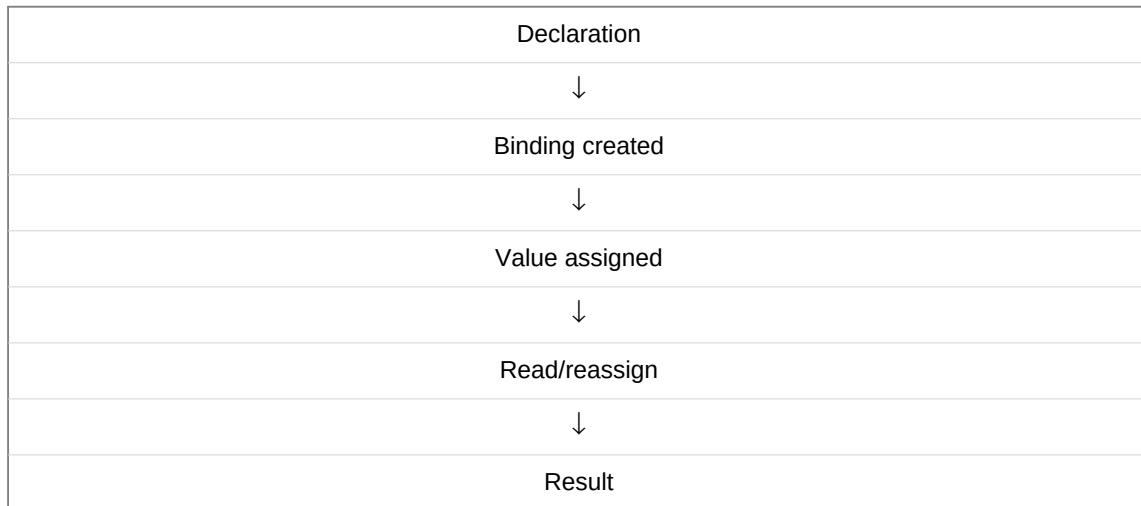
2. Variables

Definition & Theory

A variable is a named binding that lets a program refer to a value. Modern JavaScript mainly uses `let` and `const`; `var` has older function-scoping behavior.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
let count = 0;
count++;
const name = "Sai";
console.log(count, name);
```

Conclusion

Use `const` when the binding should not be reassigned and `let` when reassignment is required.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

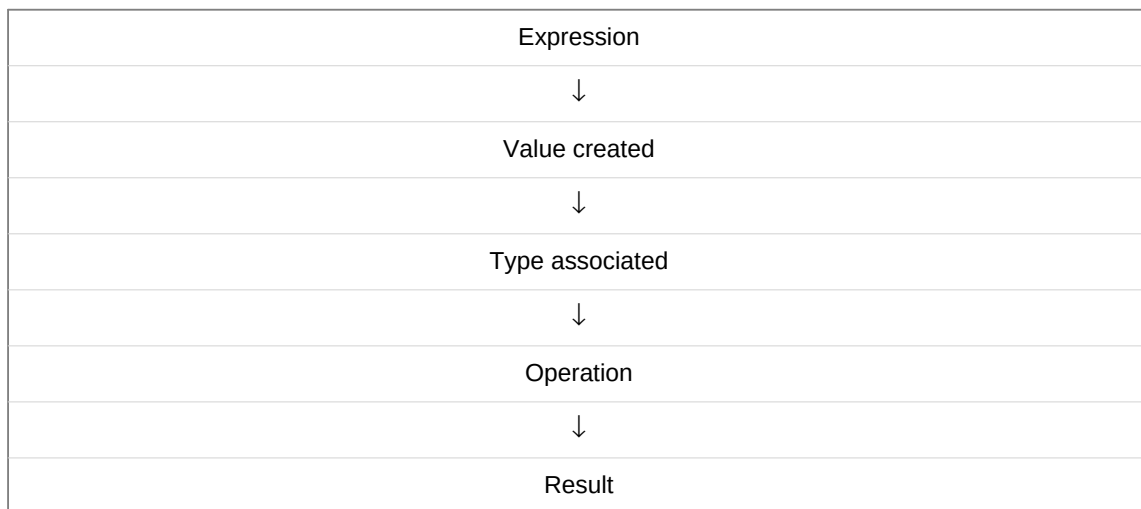
3. Data Types

Definition & Theory

JavaScript values have types. Important primitive types include string, number, bigint, boolean, undefined, null, and symbol; objects are non-primitive values.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const name = "Sai";
const age = 20;
const active = true;

console.log(typeof name);
console.log(typeof age);
console.log(typeof active);
```

Conclusion

JavaScript is dynamically typed: variables can refer to values of different types at different times.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

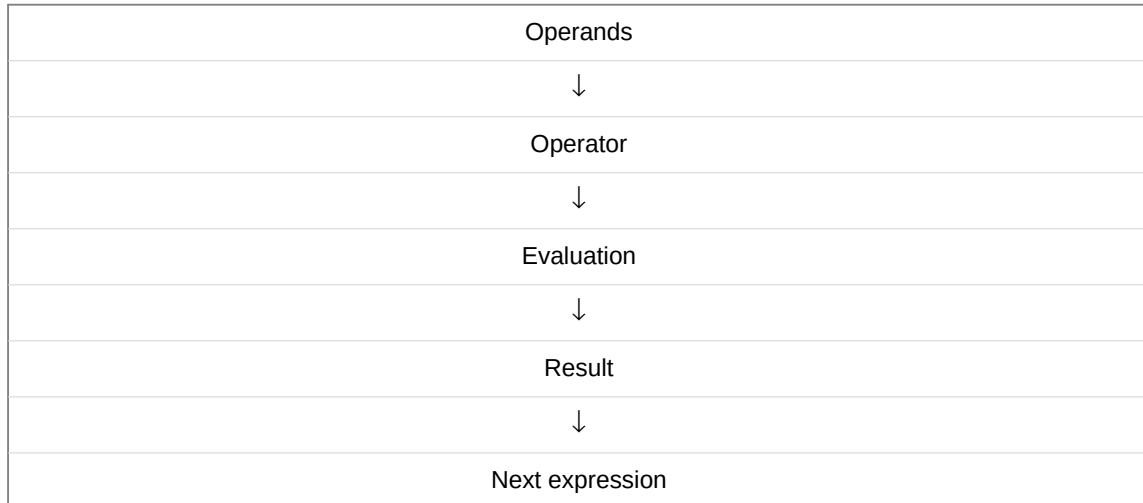
4. Operators

Definition & Theory

Operators perform arithmetic, comparison, assignment, logical, and conditional operations.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const a = 10;
const b = 3;

console.log(a + b);
console.log(a > b);
console.log(a % b);
console.log(a > 5 && b < 5);
```

Conclusion

Expressions are built from values and operators. Parentheses can make complex expressions clearer.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

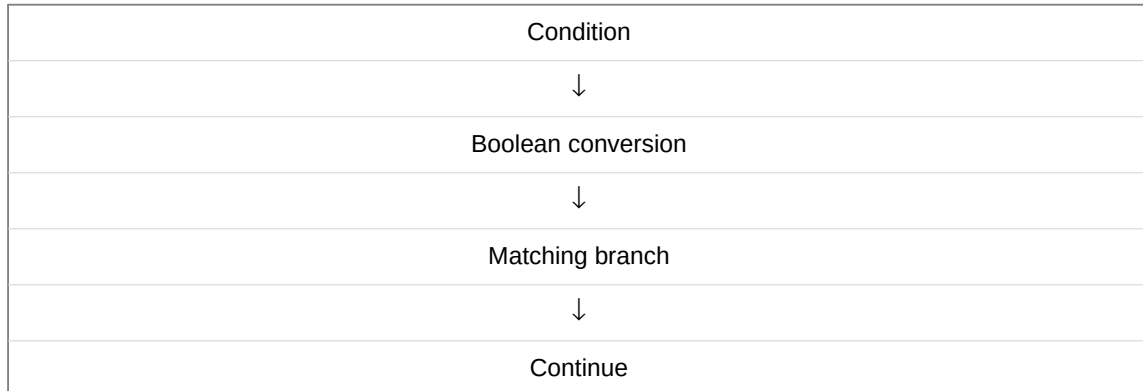
5. if / else

Definition & Theory

Conditionals allow a program to choose different code paths based on conditions.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const score = 82;

if (score >= 90) {
  console.log("A");
} else if (score >= 75) {
  console.log("B");
} else {
  console.log("C");
}
```

Conclusion

Only the selected branch executes. Conditions are evaluated from top to bottom in an if/else-if chain.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

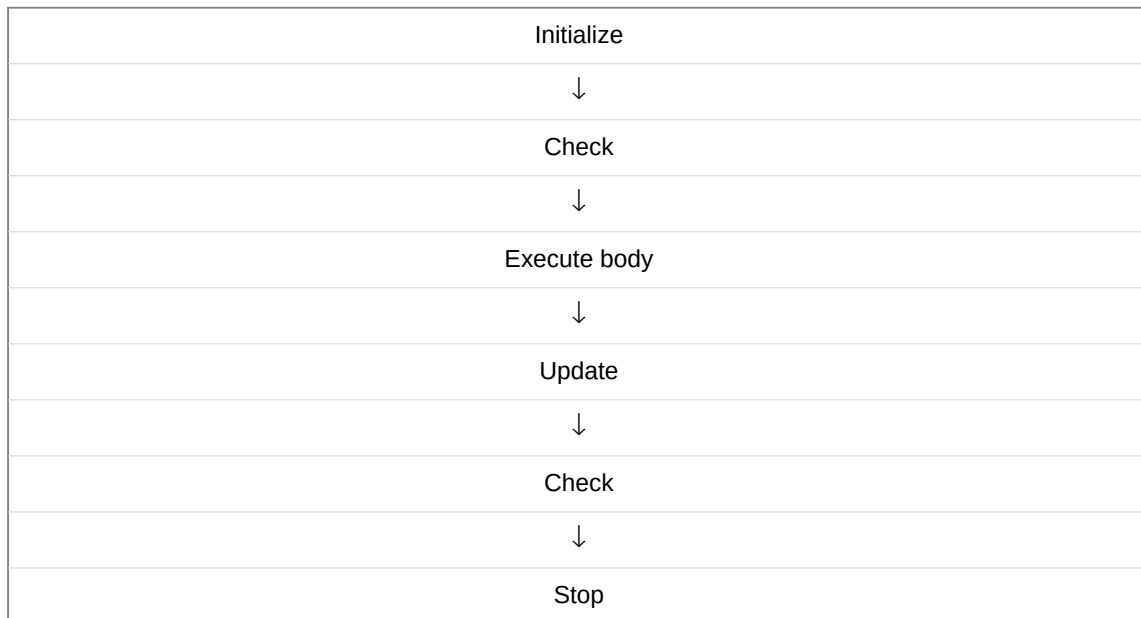
6. Loops

Definition & Theory

Loops repeat code while a condition remains true or while an iterable has more values.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

Conclusion

A loop needs a clear stopping condition. An incorrect update can create an infinite loop.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

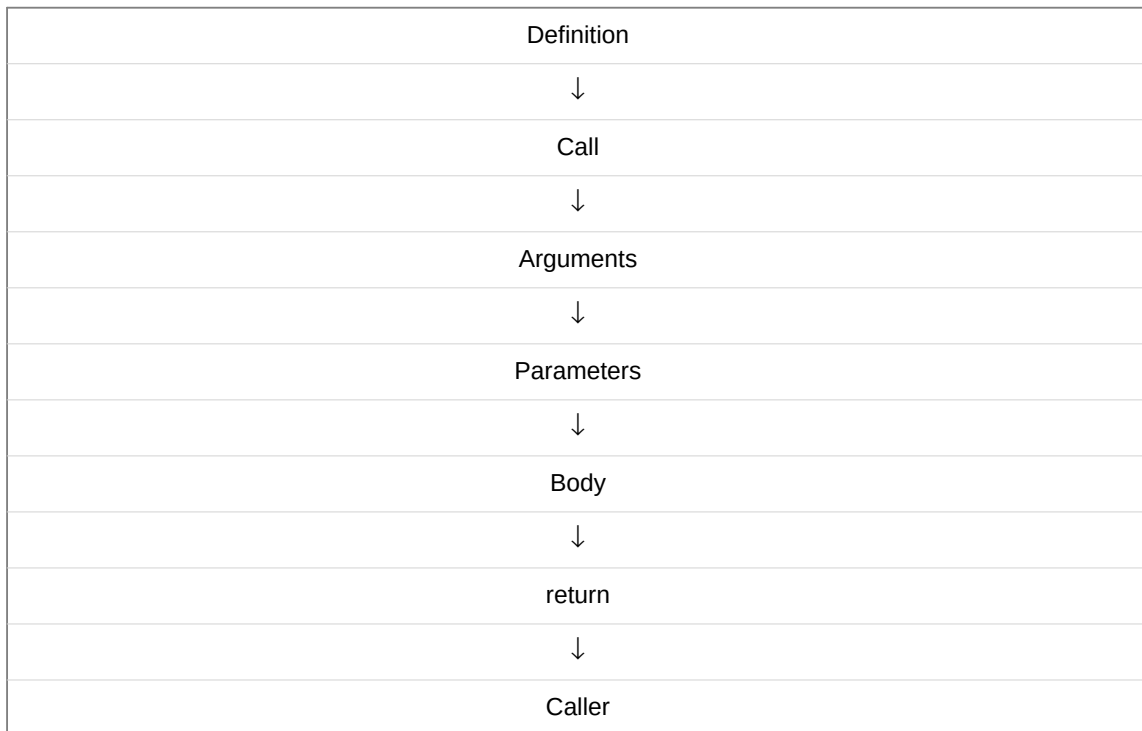
7. Functions

Definition & Theory

A function is reusable code that can receive input through parameters and return a result.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(10, 20);  
console.log(result);
```

Conclusion

Functions reduce repetition and create reusable units of behavior.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

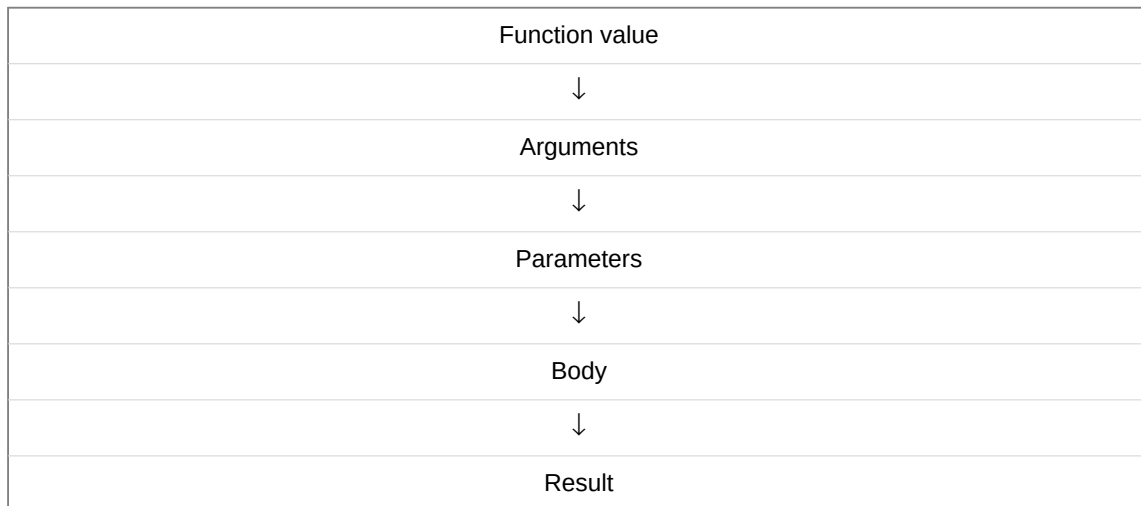
8. Arrow Functions

Definition & Theory

Arrow functions are concise function expressions and use lexical this.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const add = (a, b) => a + b;

const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);

console.log(add(2, 3));
console.log(doubled);
```

Conclusion

Arrow functions are frequently used as callbacks and short transformation functions.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

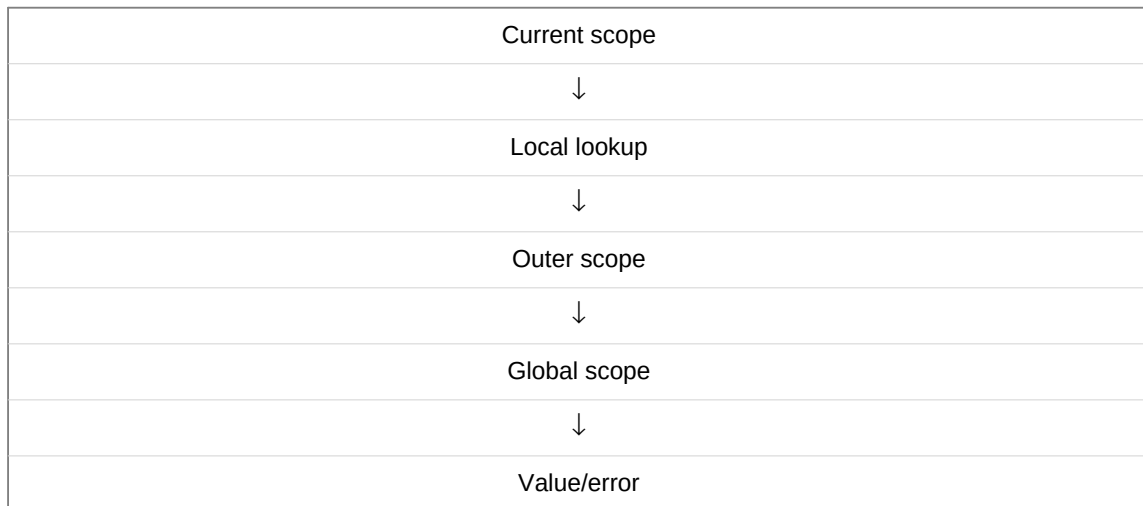
9. Scope

Definition & Theory

Scope determines where a variable can be accessed. JavaScript has global, function, and block scopes.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const x = 10;

function test() {
  const y = 20;
  console.log(x, y);
}

test();
```

Conclusion

JavaScript uses lexical scope, meaning the structure of the source code determines accessible outer variables.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

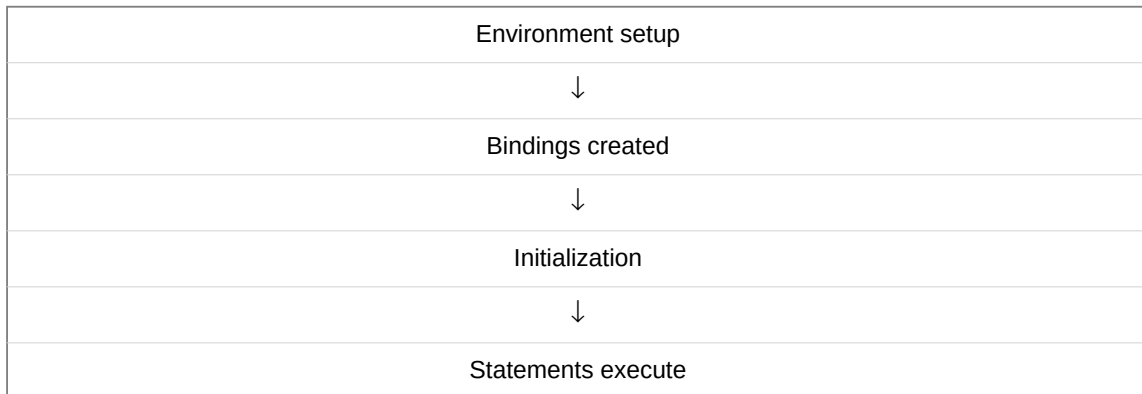
10. Hoisting & TDZ

Definition & Theory

JavaScript prepares declarations during environment setup. `let` and `const` are inaccessible before initialization because of the temporal dead zone.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
console.log(a); // undefined
var a = 10;

// let b = 20;
// Accessing b before its declaration/initialization throws.
const c = 30;
```

Conclusion

Hoisting does not mean assignments move upward. A good practice is to declare variables before using them.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

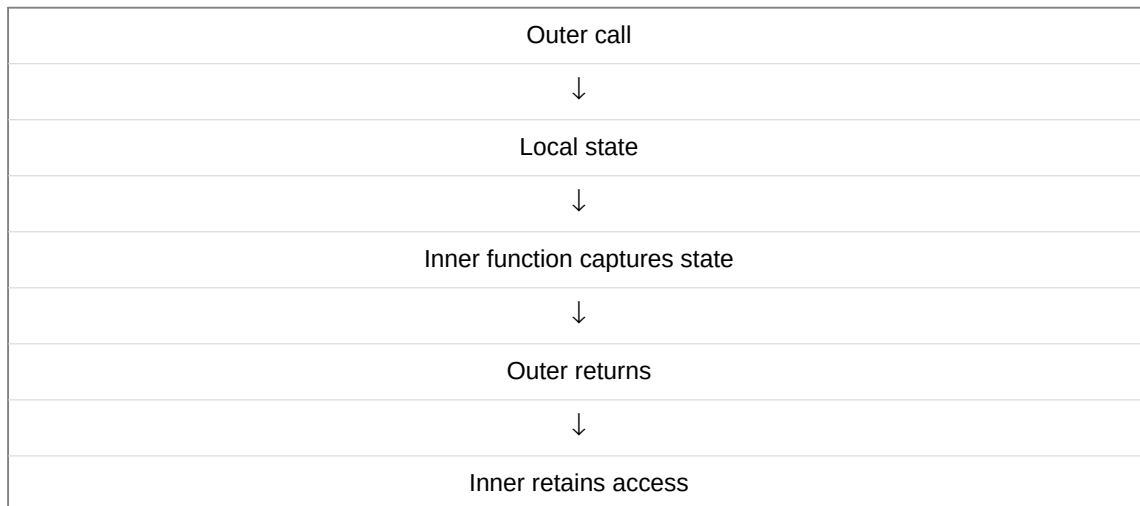
11. Closures

Definition & Theory

A closure occurs when a function retains access to variables from its surrounding lexical environment.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
function counter() {  
  let count = 0;  
  
  return () => ++count;  
}  
  
const next = counter();  
console.log(next());  
console.log(next());
```

Conclusion

Closures are useful for factories, callbacks, and maintaining private-like state.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

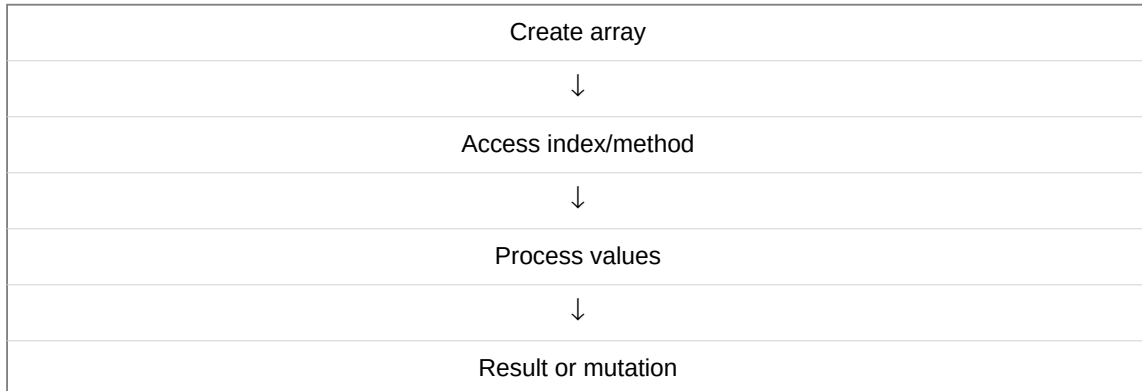
12. Arrays

Definition & Theory

Arrays are ordered, zero-indexed collections of values.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const fruits = ["apple", "banana"];

fruits.push("mango");
console.log(fruits);
console.log(fruits[0]);
```

Conclusion

Arrays are heavily used for collections and are supported by many higher-order methods.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

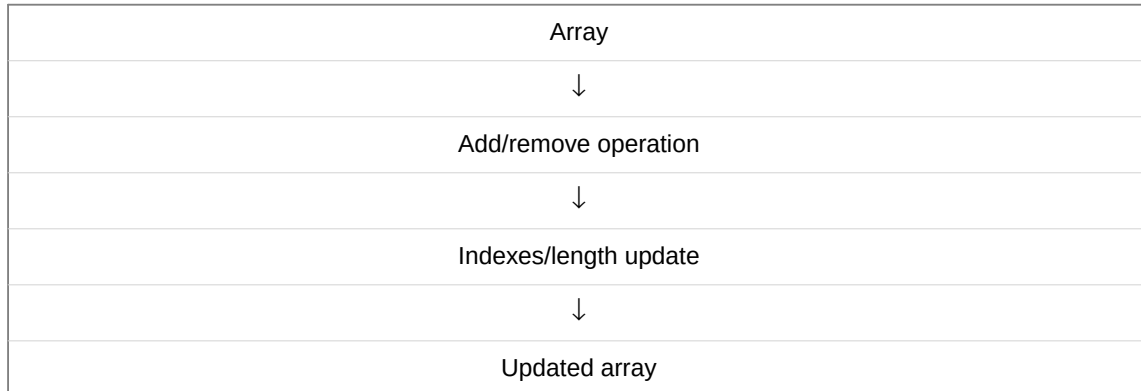
13. push / pop / shift / unshift

Definition & Theory

push and pop operate at the end of an array; shift and unshift operate at the beginning.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const items = [1, 2, 3];

items.push(4);
items.pop();
items.unshift(0);
items.shift();

console.log(items);
```

Conclusion

These methods mutate the original array. Know which end of the array each method affects.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

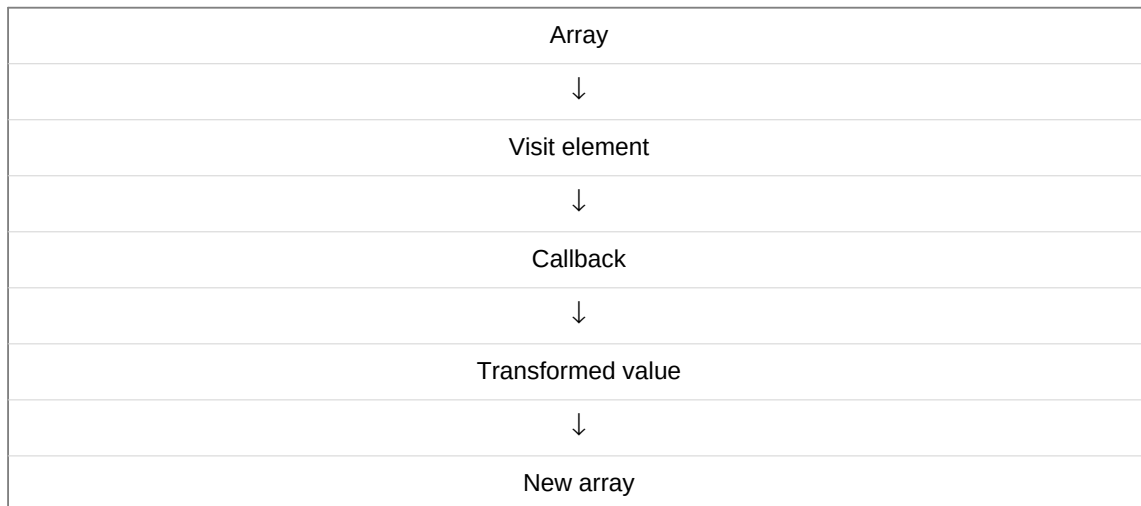
14. map()

Definition & Theory

map transforms each element and returns a new array containing the callback results.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const numbers = [1, 2, 3, 4];  
  
const doubled = numbers.map(n => n * 2);  
  
console.log(doubled);
```

Conclusion

map normally produces one output for every input element.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

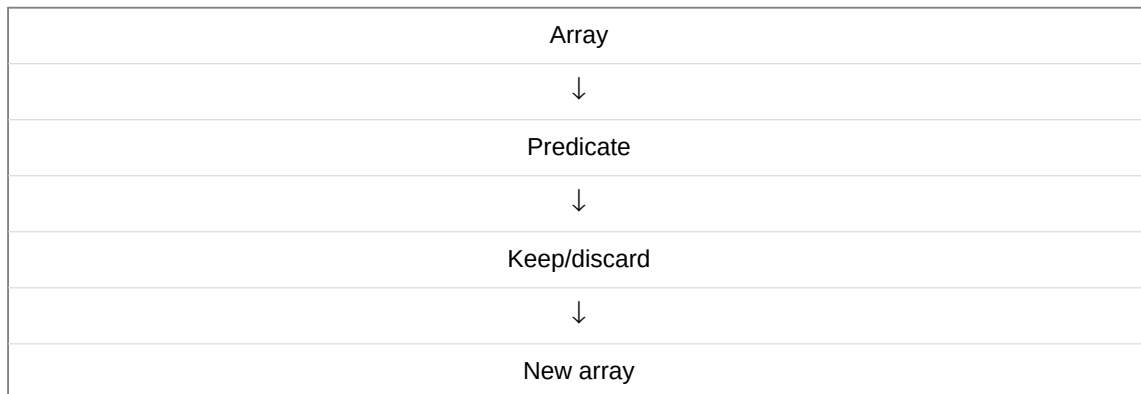
15. filter()

Definition & Theory

filter selects elements whose callback returns true and returns them in a new array.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const numbers = [1, 2, 3, 4, 5, 6];

const even = numbers.filter(n => n % 2 === 0);

console.log(even);
```

Conclusion

Use filter when the goal is selection rather than transformation.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

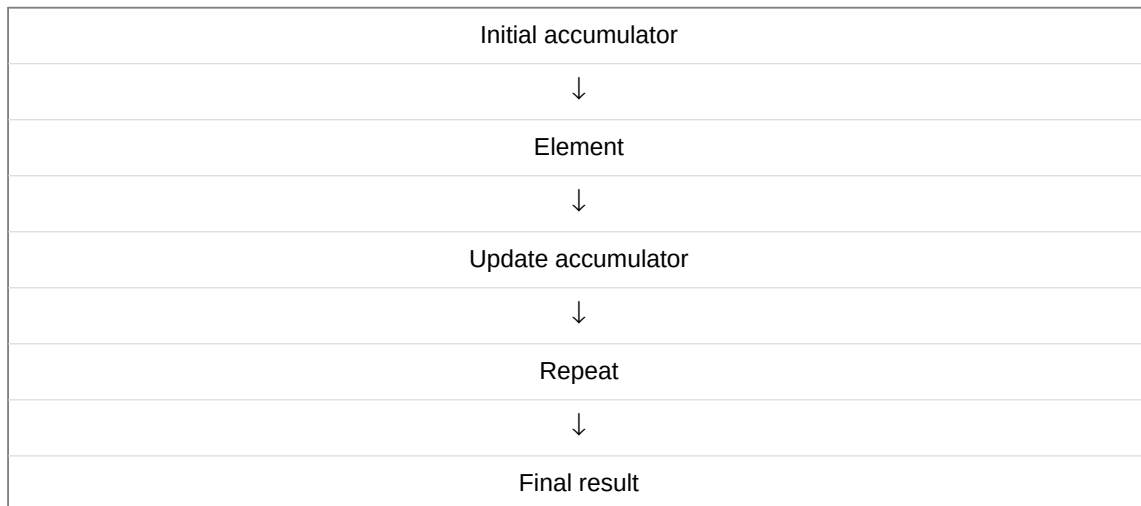
16. reduce()

Definition & Theory

reduce combines array elements into one accumulated result such as a sum, object, or grouped value.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const numbers = [10, 20, 30];

const total = numbers.reduce(
  (sum, n) => sum + n,
  0
);

console.log(total);
```

Conclusion

The accumulator is the key idea. Always understand its initial value and how each callback updates it.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

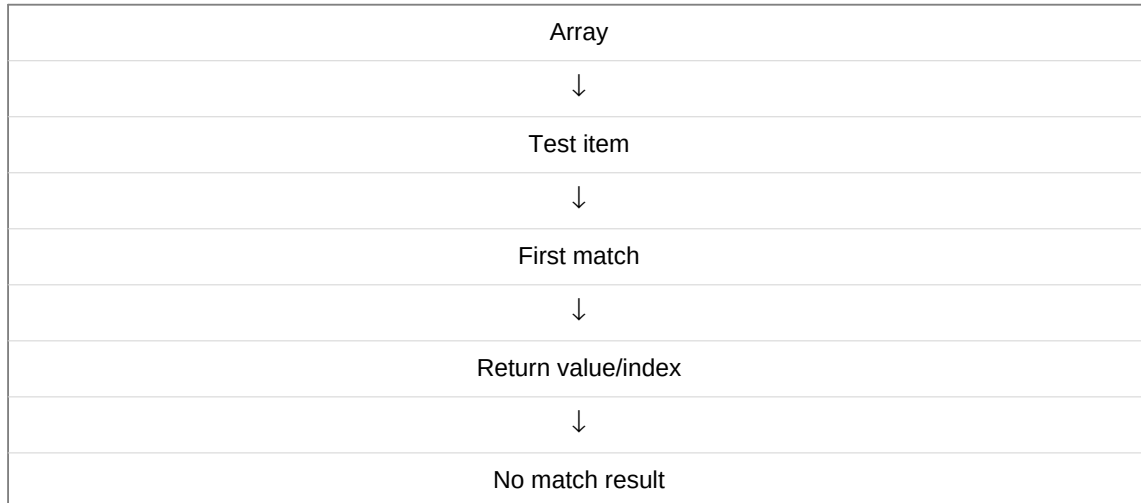
17. find() / findIndex()

Definition & Theory

find returns the first matching element; findIndex returns the index of the first matching element.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const users = [{ id: 1 }, { id: 2 }];

const user = users.find(u => u.id === 2);
const index = users.findIndex(u => u.id === 2);

console.log(user);
console.log(index);
```

Conclusion

find returns undefined when no item matches; findIndex returns -1.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

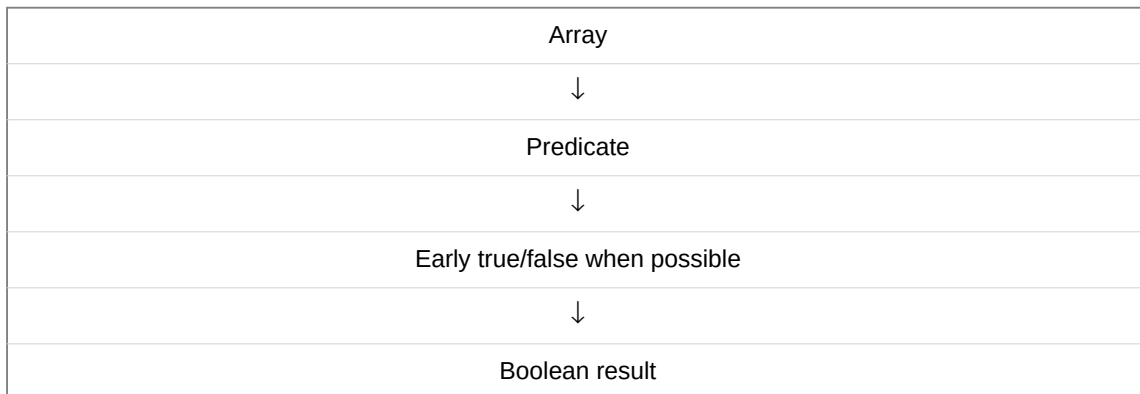
18. some() / every()

Definition & Theory

`some` checks whether at least one element passes a test. `every` checks whether all elements pass.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const ages = [18, 21, 25];

console.log(ages.some(a => a < 20));
console.log(ages.every(a => a >= 18));
```

Conclusion

Both methods answer yes/no questions about a collection and return booleans.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

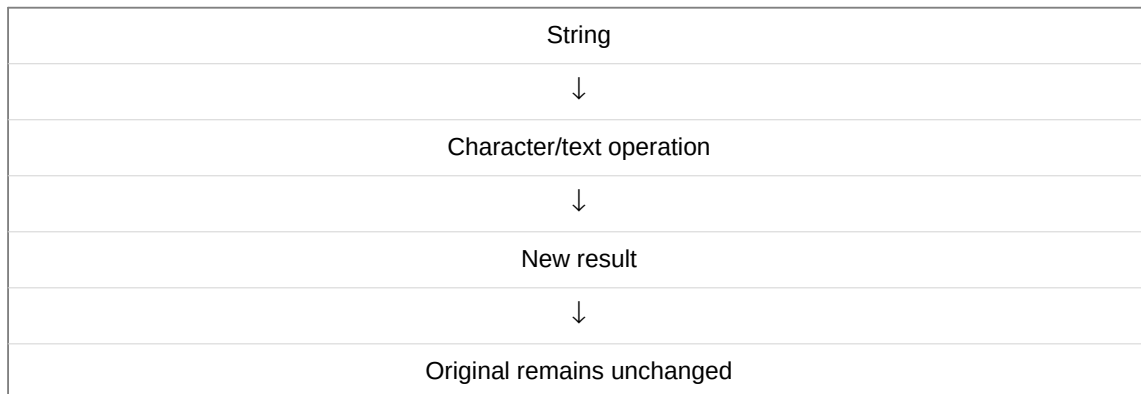
19. Strings

Definition & Theory

Strings represent text and are immutable values.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const text = "JavaScript";

console.log(text.length);
console.log(text[0]);
console.log(text.includes("Script"));
```

Conclusion

String operations return new values; they do not modify the original string.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

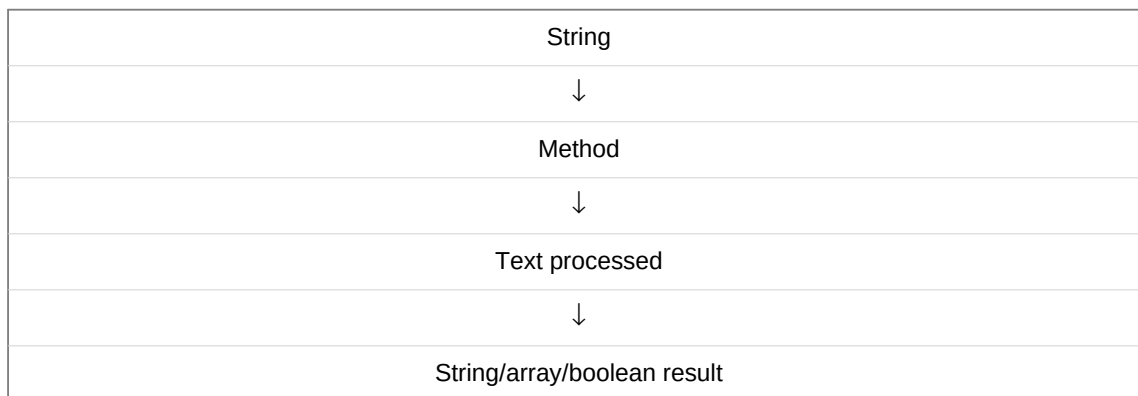
20. String Methods

Definition & Theory

JavaScript provides methods for searching, slicing, splitting, replacing, trimming, and changing case.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const text = " JavaScript ";

console.log(text.trim());
console.log(text.slice(1, 5));
console.log(text.toUpperCase());
console.log(text.includes("Script"));
```

Conclusion

Learn both what each method does and what type it returns.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

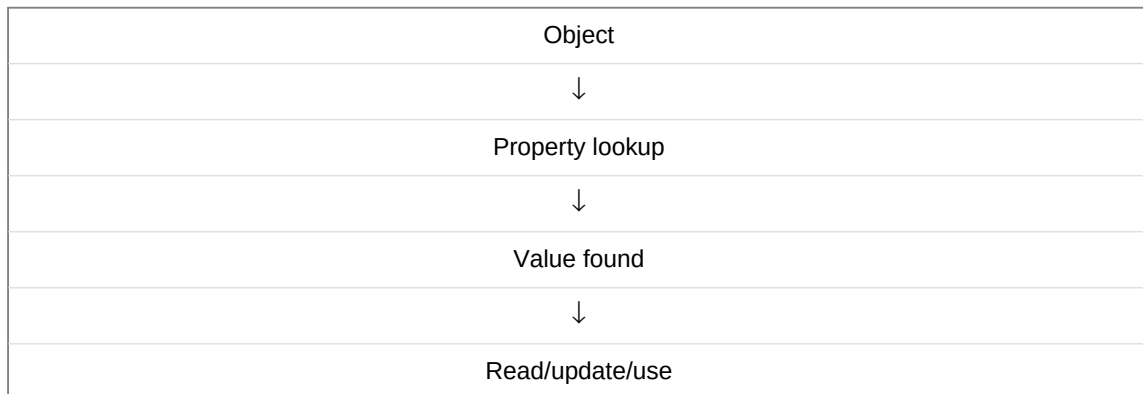
21. Objects

Definition & Theory

Objects store related data as key-value properties and can also contain methods.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = {  
  name: "Sai",  
  age: 20  
};  
  
console.log(user.name);  
user.age = 21;  
console.log(user);
```

Conclusion

Objects are fundamental for representing users, configuration, API responses, and application state.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

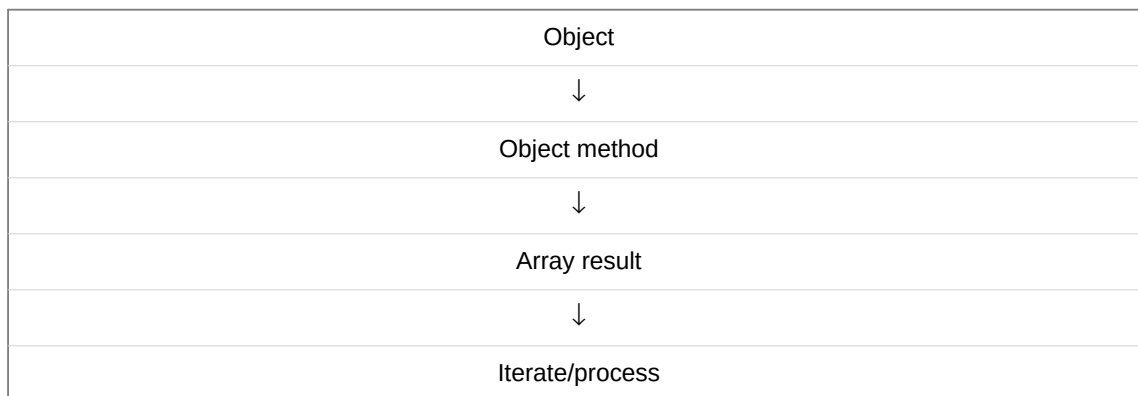
22. Object.keys / values / entries

Definition & Theory

These methods convert an object's enumerable properties into arrays of keys, values, or key-value pairs.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = { name: "Sai", age: 20 };

console.log(Object.keys(user));
console.log(Object.values(user));
console.log(Object.entries(user));
```

Conclusion

These methods are useful when you need array operations on object data.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

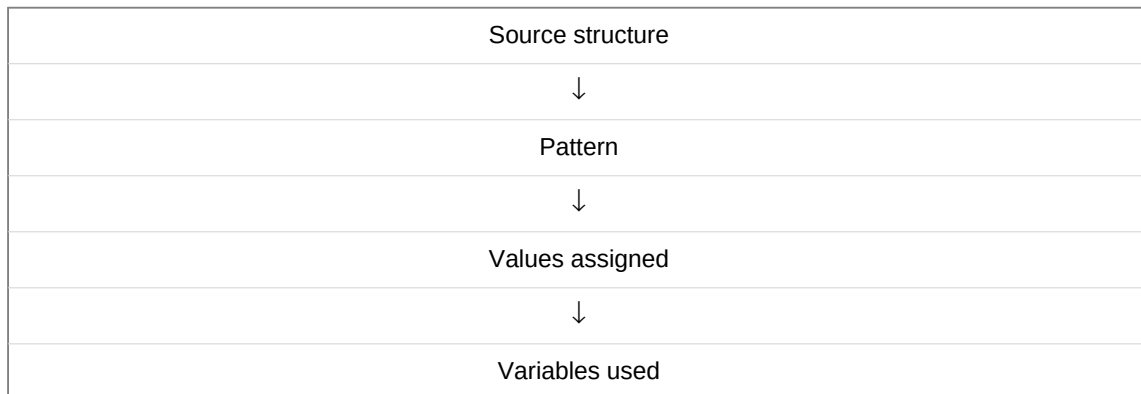
23. Destructuring

Definition & Theory

Destructuring extracts values from arrays or properties from objects into variables.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = { name: "Sai", age: 20 };

const { name, age } = user;
const [first, second] = [10, 20];

console.log(name, age);
console.log(first, second);
```

Conclusion

Destructuring makes data extraction concise and is common in modern JavaScript.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

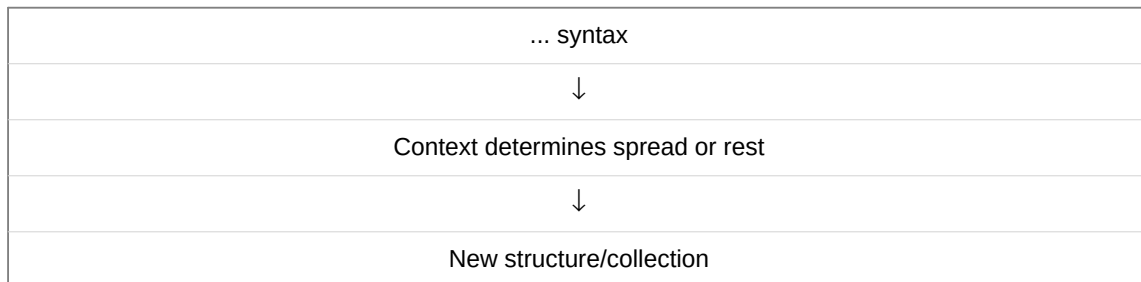
24. Spread & Rest

Definition & Theory

Spread expands values into a new structure; rest collects remaining values.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const a = [1, 2];
const b = [...a, 3];

function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}

console.log(b);
console.log(sum(1, 2, 3));
```

Conclusion

Spread and rest share syntax but have opposite purposes.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

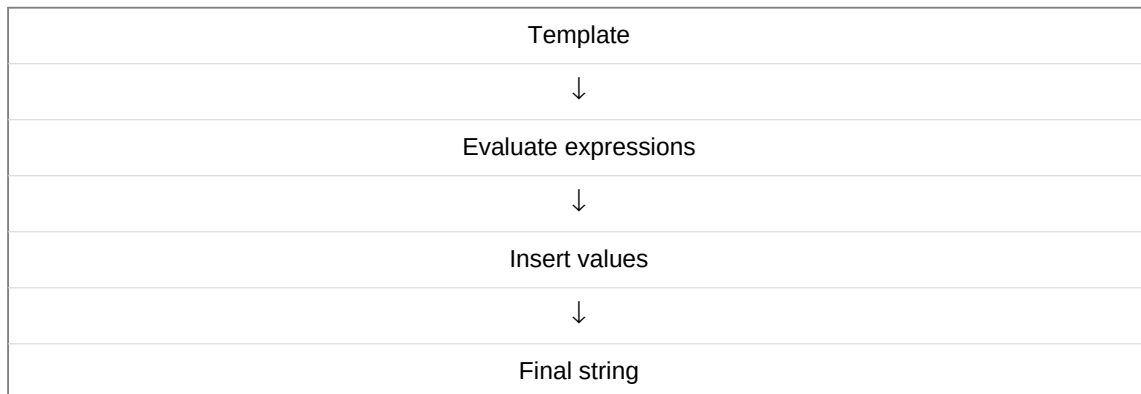
25. Template Literals

Definition & Theory

Template literals use backticks and allow expressions inside `${...}`.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const name = "Sai";
const score = 95;

const message = `Hello ${name}, score: ${score}`;

console.log(message);
```

Conclusion

They are convenient for dynamic strings and multiline text.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

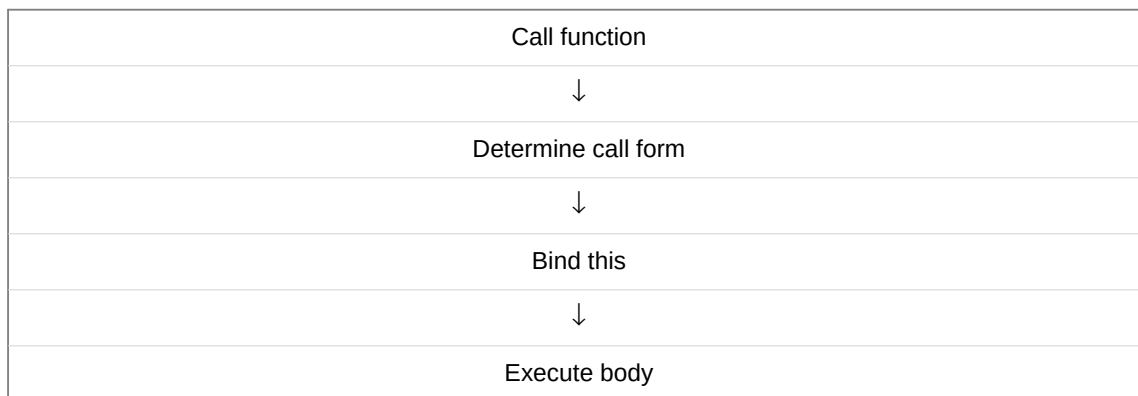
26. this

Definition & Theory

this is a context value determined by the invocation of a function; arrow functions inherit this lexically.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = {  
  name: "Sai",  
  greet() {  
    console.log(this.name);  
  }  
};  
  
user.greet();
```

Conclusion

When debugging this, inspect how the function is called.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

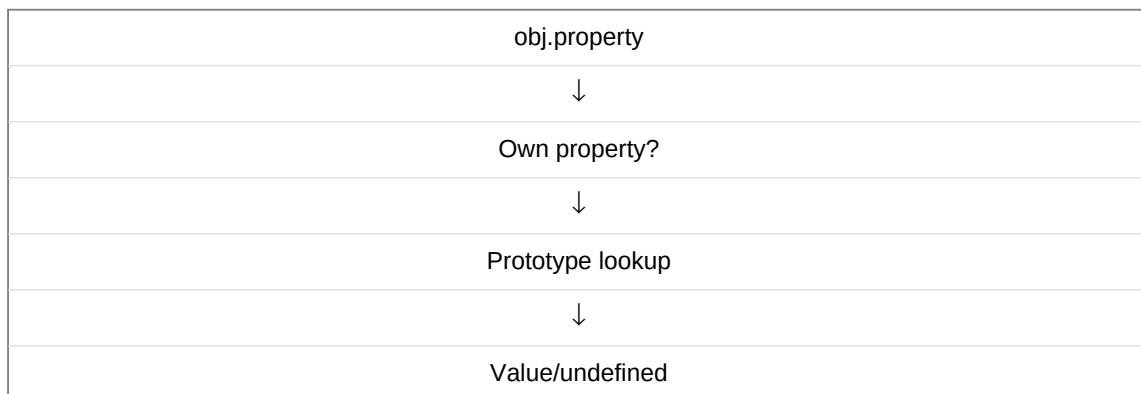
27. Prototypes

Definition & Theory

Objects can inherit properties through a prototype chain. Property lookup continues through that chain when needed.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const animal = {  
  speak() {  
    console.log("sound");  
  }  
};  
  
const dog = Object.create(animal);  
dog.speak();
```

Conclusion

Prototype inheritance is part of JavaScript's core object model.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

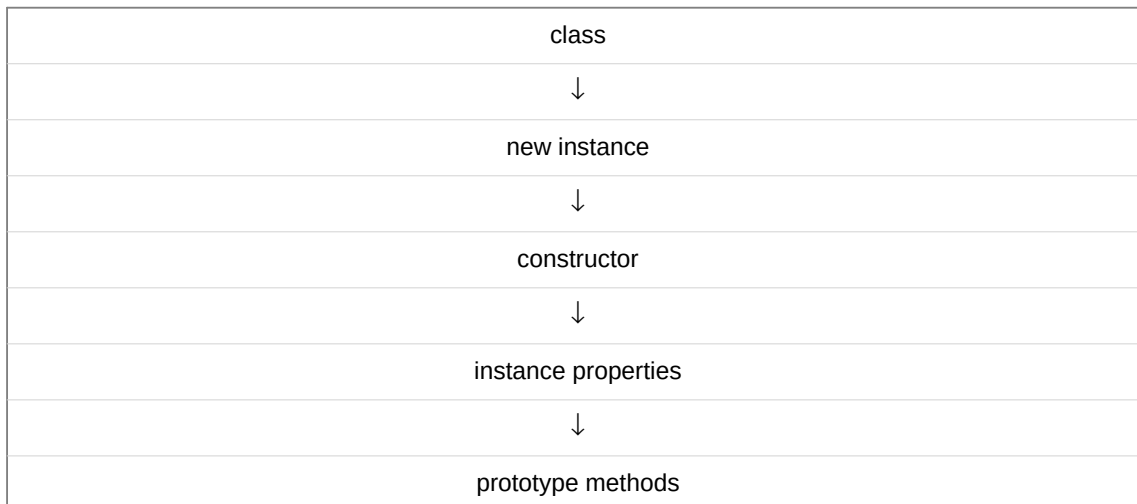
28. Classes

Definition & Theory

Classes provide syntax for constructors and methods while using the prototype system underneath.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hi ${this.name}`;  
  }  
}  
  
const user = new User("Sai");  
console.log(user.greet());
```

Conclusion

Classes make object-oriented patterns easier to express but do not replace JavaScript's prototype model.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

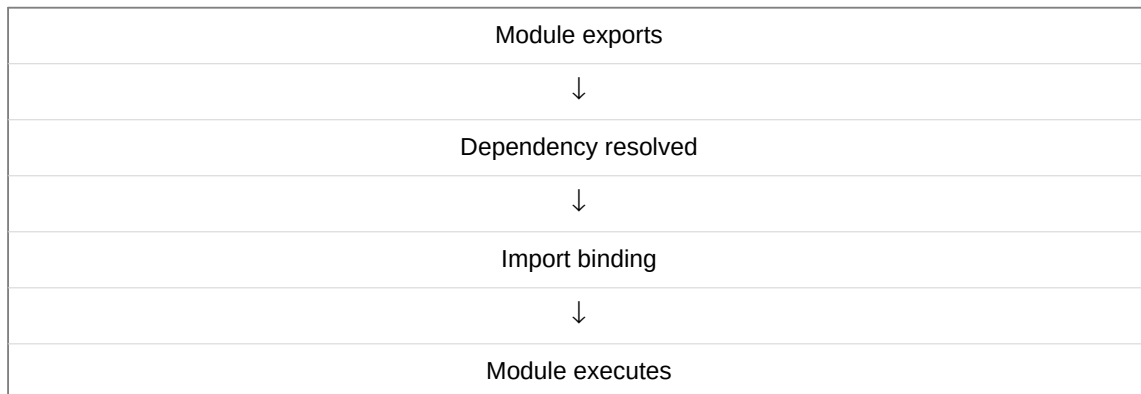
29. Modules

Definition & Theory

ES modules organize code across files using export and import.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
// math.js
export const add = (a, b) => a + b;

// app.js
import { add } from "./math.js";

console.log(add(2, 3));
```

Conclusion

Modules make dependencies explicit and reduce reliance on global variables.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

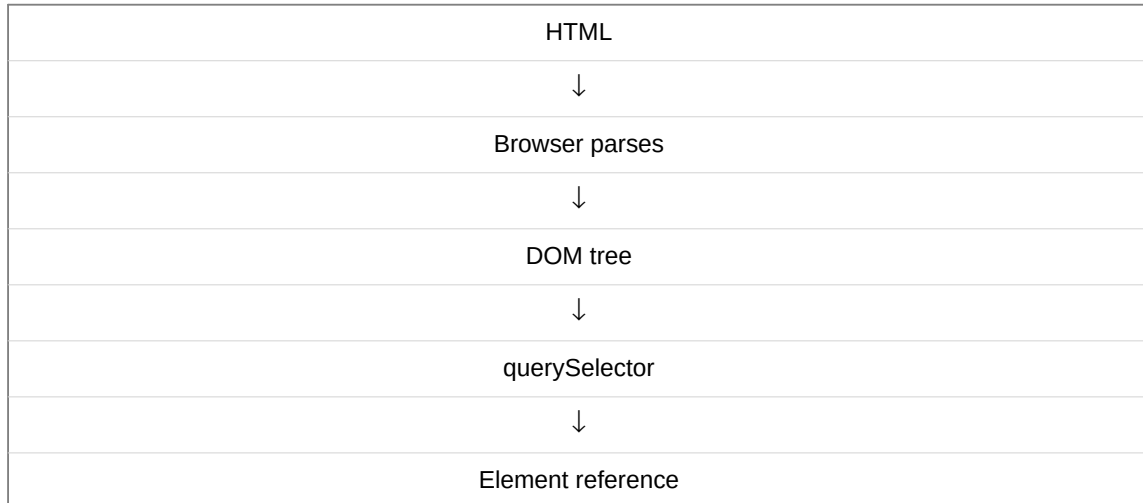
30. DOM Selection

Definition & Theory

The DOM is the browser's object representation of HTML. JavaScript can locate nodes with DOM APIs.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const heading = document.querySelector("h1");

heading.textContent = "Hello JavaScript";
heading.classList.add("active");
```

Conclusion

DOM APIs are provided by the browser environment rather than by the JavaScript language itself.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

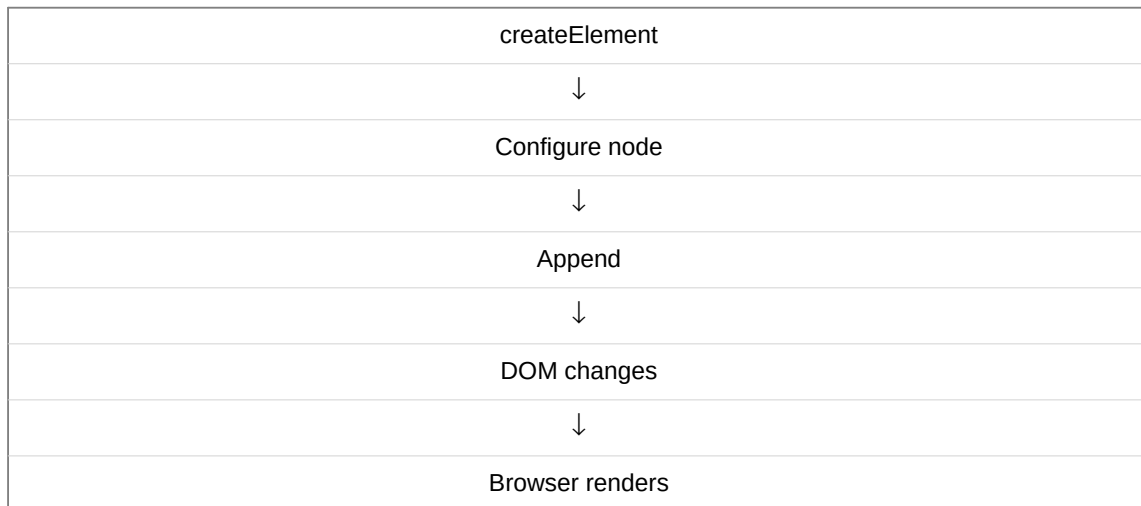
31. DOM Creation

Definition & Theory

JavaScript can create elements, configure them, and attach them to the document.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const p = document.createElement("p");

p.textContent = "New paragraph";
p.classList.add("message");

document.body.append(p);
```

Conclusion

Creating nodes programmatically is useful for dynamic interfaces.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

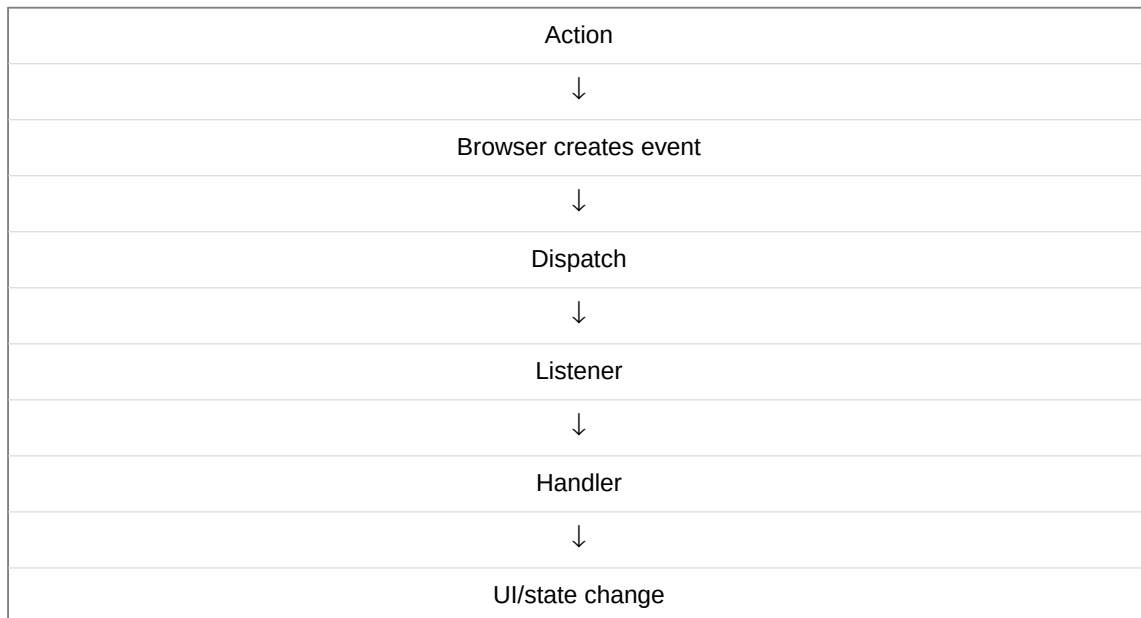
32. Events

Definition & Theory

Events represent actions such as clicks, typing, loading, and form submission.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const button = document.querySelector("button");

button.addEventListener("click", event => {
  console.log("Clicked:", event.target);
});
```

Conclusion

Event listeners let applications react to user and browser activity.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

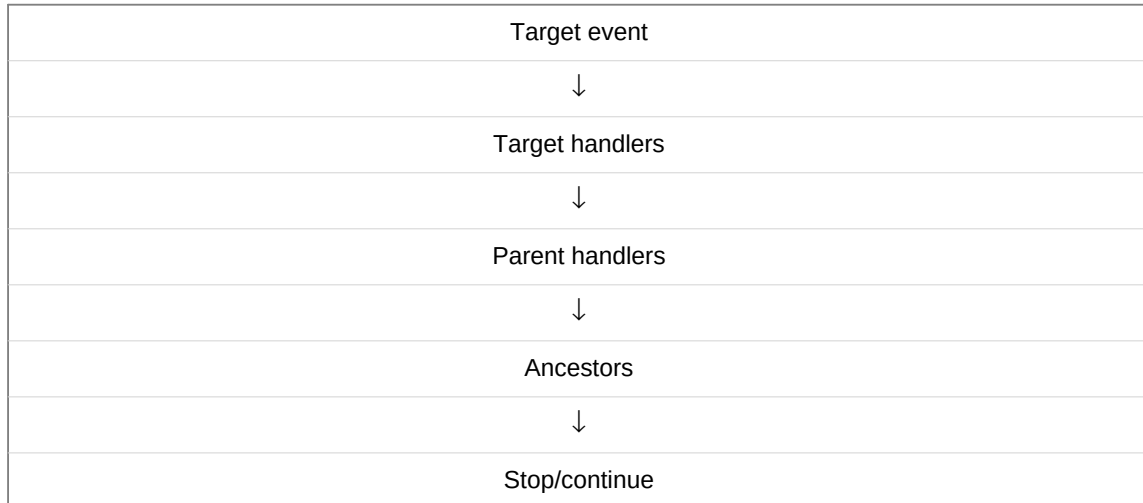
33. Event Bubbling

Definition & Theory

Many DOM events travel from the target element toward ancestor elements after reaching the target.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
document.querySelector("div").addEventListener("click", e => {  
  console.log("parent received", e.target);  
});
```

Conclusion

Bubbling is important for event delegation and understanding why parent listeners can react to child events.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

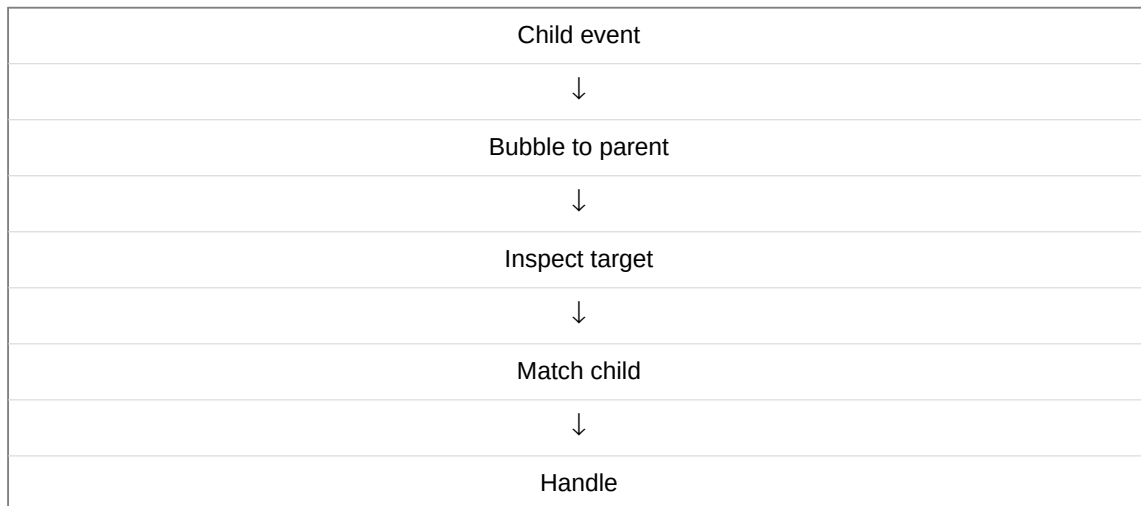
34. Event Delegation

Definition & Theory

Event delegation uses a parent listener to handle events from matching child elements.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
document.querySelector("ul").addEventListener("click", e => {  
  if (e.target.matches("li")) {  
    console.log(e.target.textContent);  
  }  
});
```

Conclusion

Delegation can simplify dynamic lists because one parent listener can handle many children.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

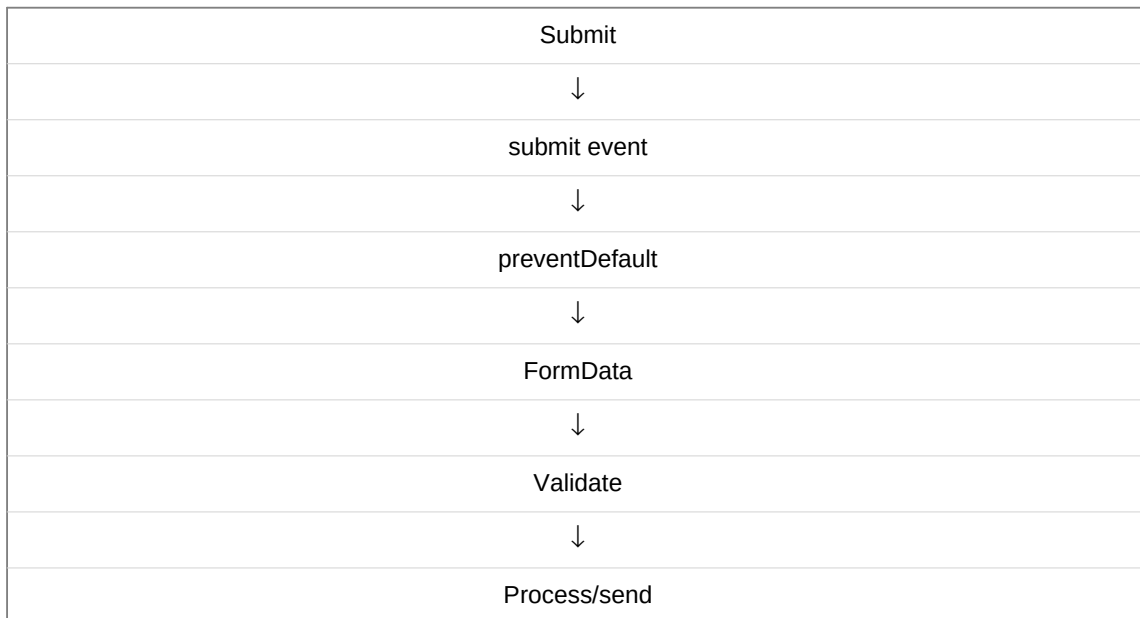
35. Forms & FormData

Definition & Theory

Forms collect user input. FormData provides an interface for reading form fields.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const form = document.querySelector("form");

form.addEventListener("submit", e => {
  e.preventDefault();

  const data = new FormData(form);
  console.log(data.get("email"));
});
```

Conclusion

Client-side validation improves user experience but should not replace server-side validation.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

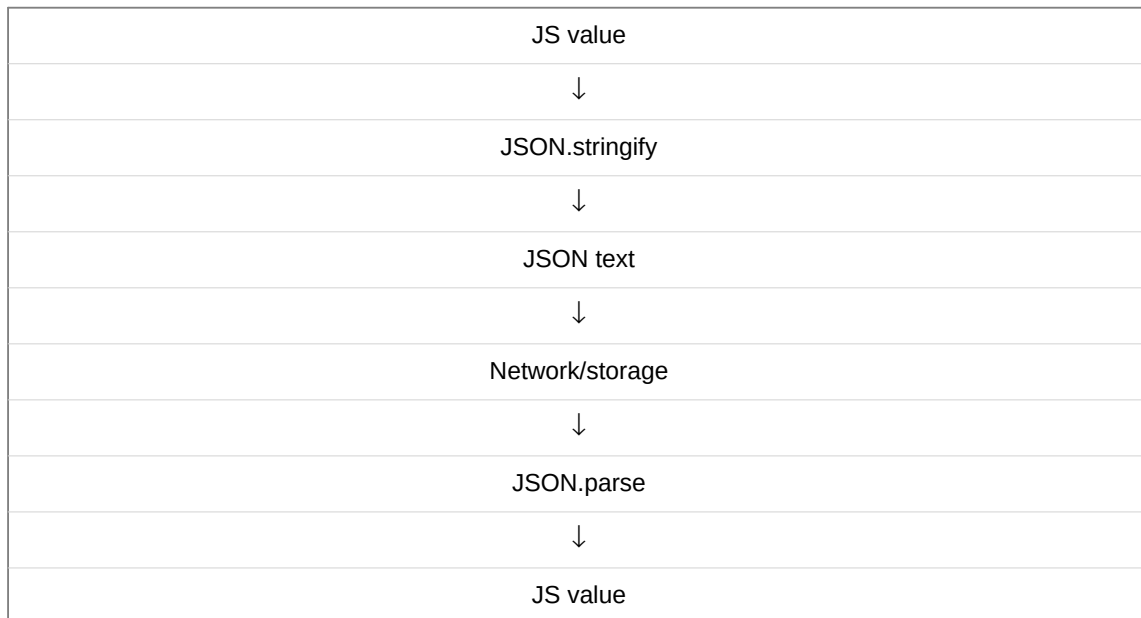
36. JSON

Definition & Theory

JSON is a text format commonly used to exchange structured data between applications.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = { name: "Sai", age: 20 };

const json = JSON.stringify(user);
const parsed = JSON.parse(json);

console.log(parsed.name);
```

Conclusion

JSON is text; JavaScript objects are runtime values.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

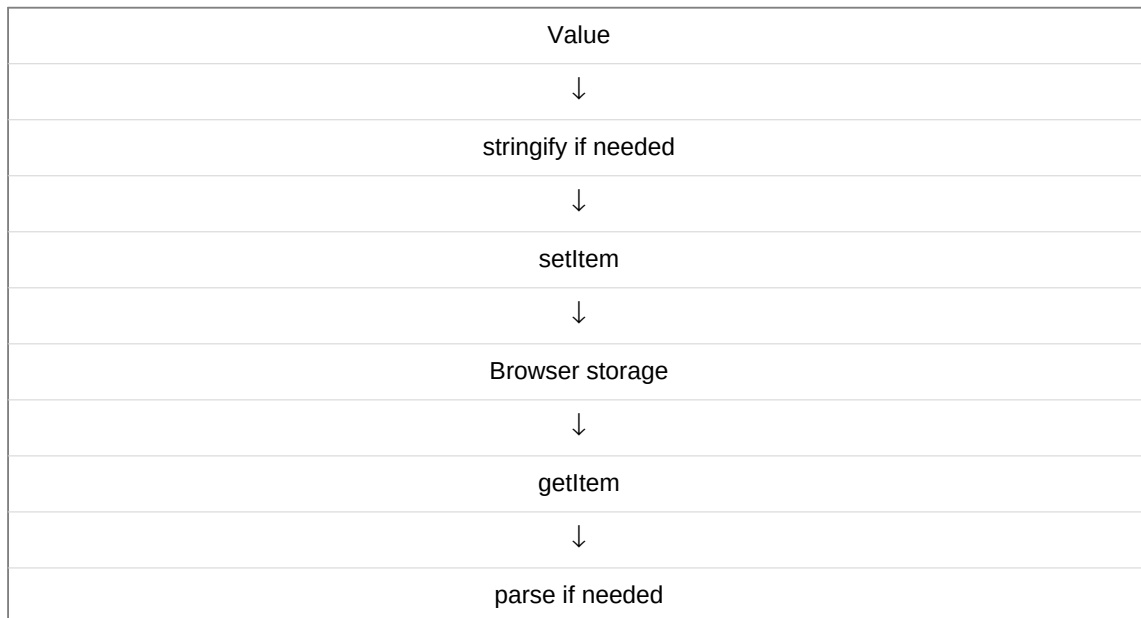
37. localStorage

Definition & Theory

localStorage persists string key-value data for a browser origin.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = { name: "Sai" };

localStorage.setItem("user", JSON.stringify(user));

const saved = JSON.parse(localStorage.getItem("user"));
console.log(saved.name);
```

Conclusion

Storage values are strings, so structured data commonly needs JSON serialization.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

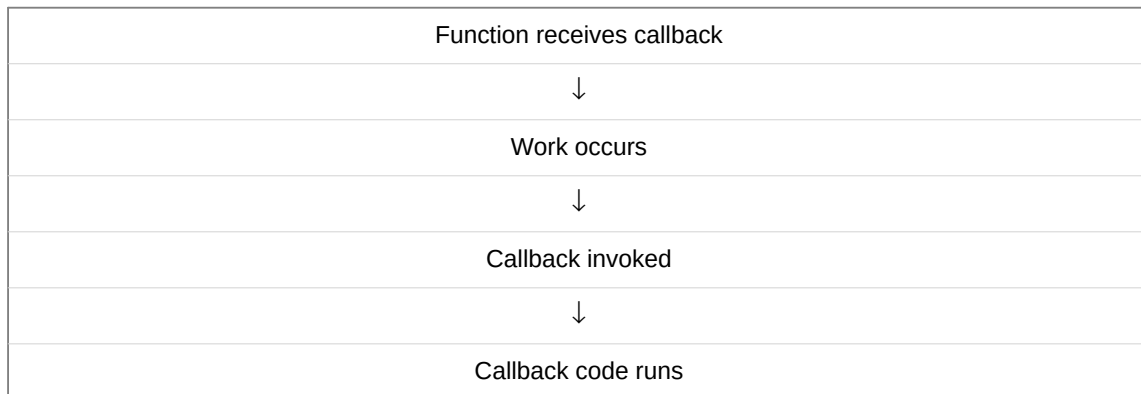
38. Callbacks

Definition & Theory

A callback is a function passed to another function so it can be invoked later or during an operation.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
function process(name, callback) {  
  console.log("Processing", name);  
  callback();  
}  
  
process("Sai", () => console.log("Done"));
```

Conclusion

Callbacks are common in array methods, event listeners, and asynchronous APIs.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

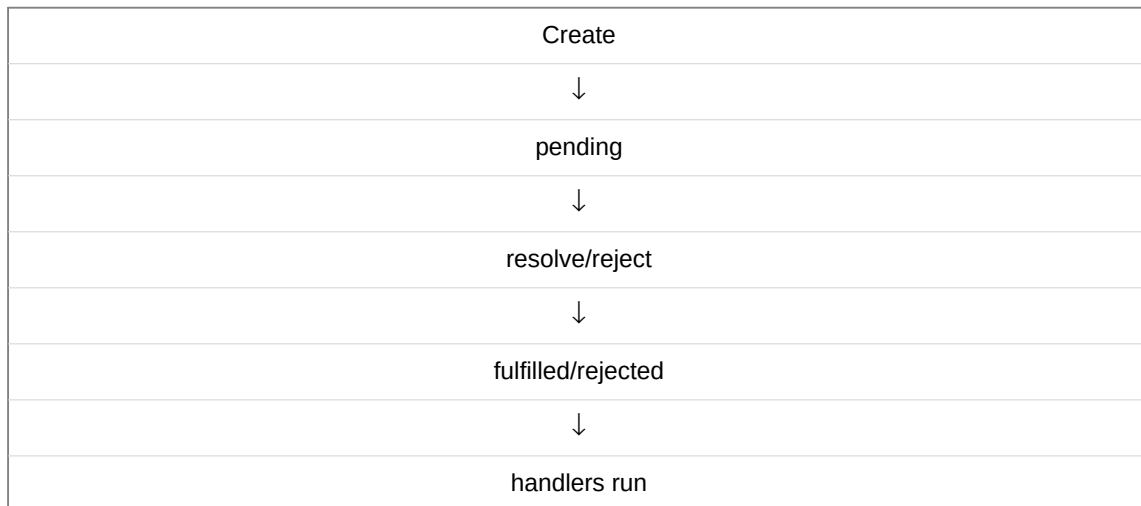
39. Promises

Definition & Theory

A Promise represents the eventual success or failure of an asynchronous operation.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const promise = new Promise(resolve => {  
  setTimeout(() => resolve("Done"), 500);  
});  
  
promise.then(result => console.log(result));
```

Conclusion

Promises make asynchronous results composable through then, catch, and finally.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

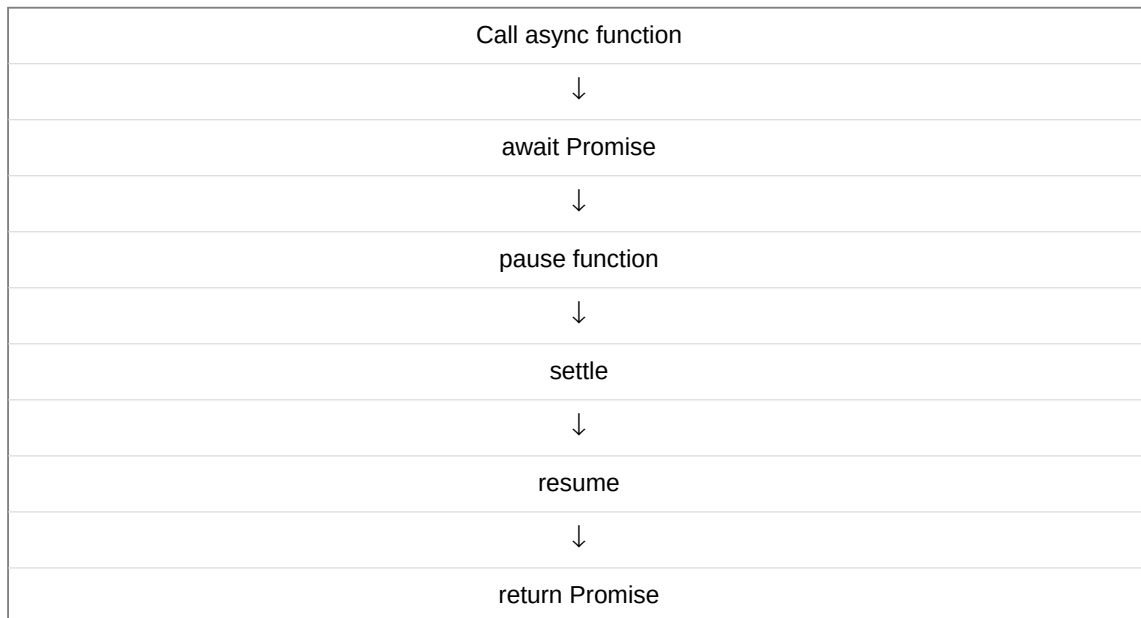
40. async / await

Definition & Theory

async functions return promises, while await pauses that async function until a promise settles.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
async function load() {  
  const response = await fetch("/api/users");  
  const users = await response.json();  
  return users;  
}  
  
load().then(console.log);
```

Conclusion

async/await changes syntax, not the underlying asynchronous nature of promises.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

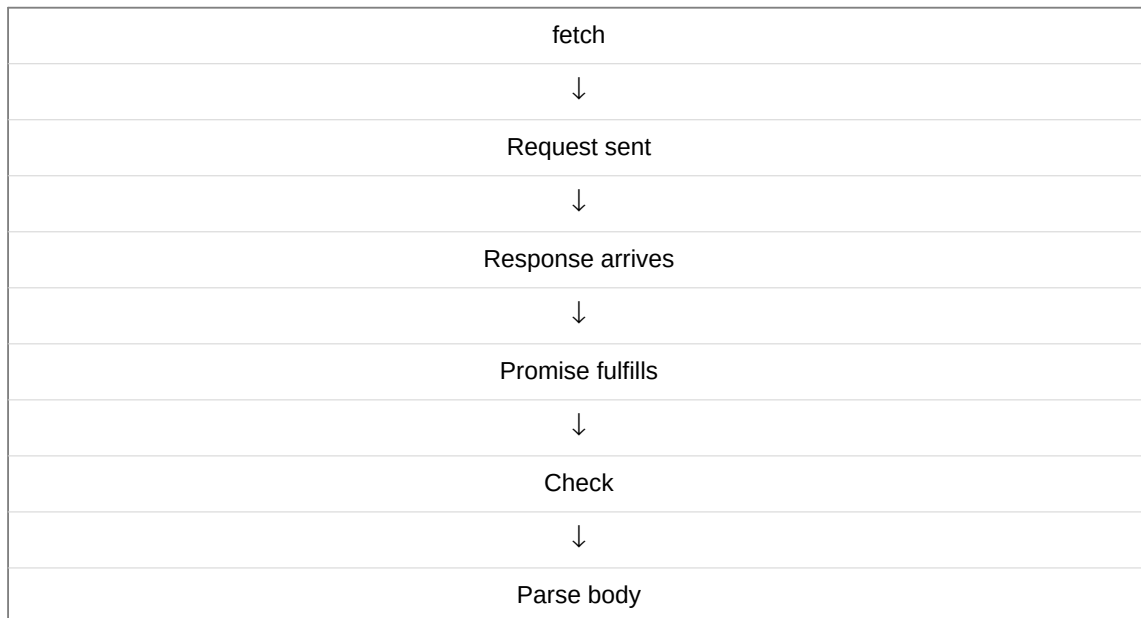
41. fetch()

Definition & Theory

fetch performs HTTP requests and returns a Promise for a Response.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
async function getData() {  
  const response = await fetch("/api/data");  
  
  if (!response.ok) {  
    throw new Error("Request failed");  
  }  
  
  return response.json();  
}
```

Conclusion

fetch does not reject simply because the server returns HTTP 404 or 500, so check response.ok.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

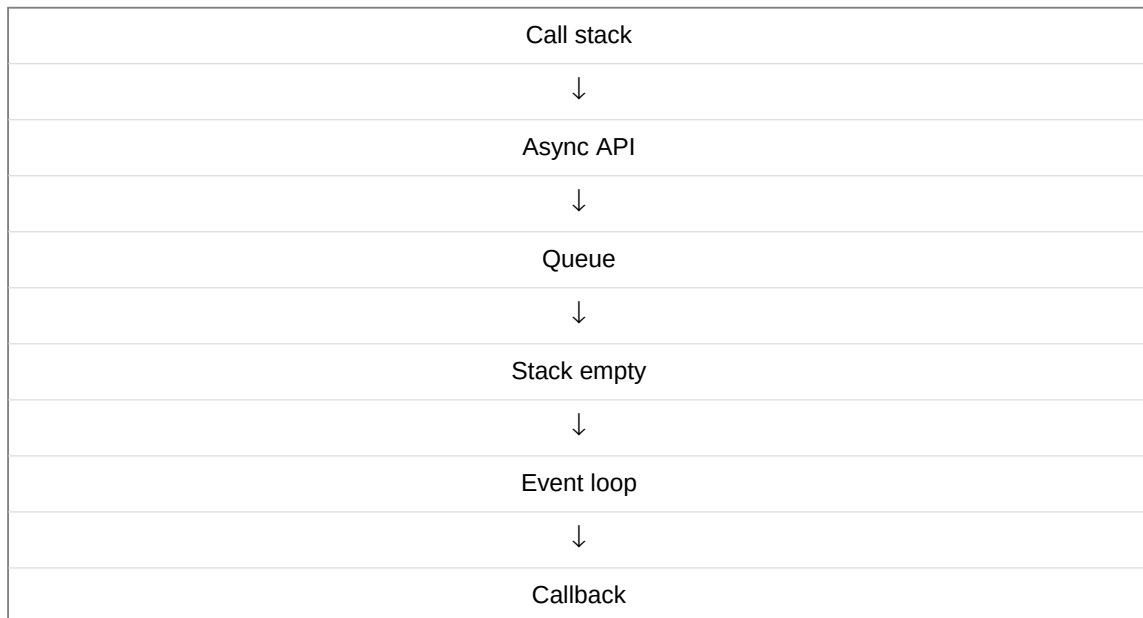
42. Event Loop

Definition & Theory

The event loop coordinates synchronous execution with queued asynchronous callbacks.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
console.log("A");

setTimeout(() => console.log("B"), 0);

console.log("C");

// A
// C
// B
```

Conclusion

The event loop is a key mental model for understanding timers, promises, and browser/server concurrency.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

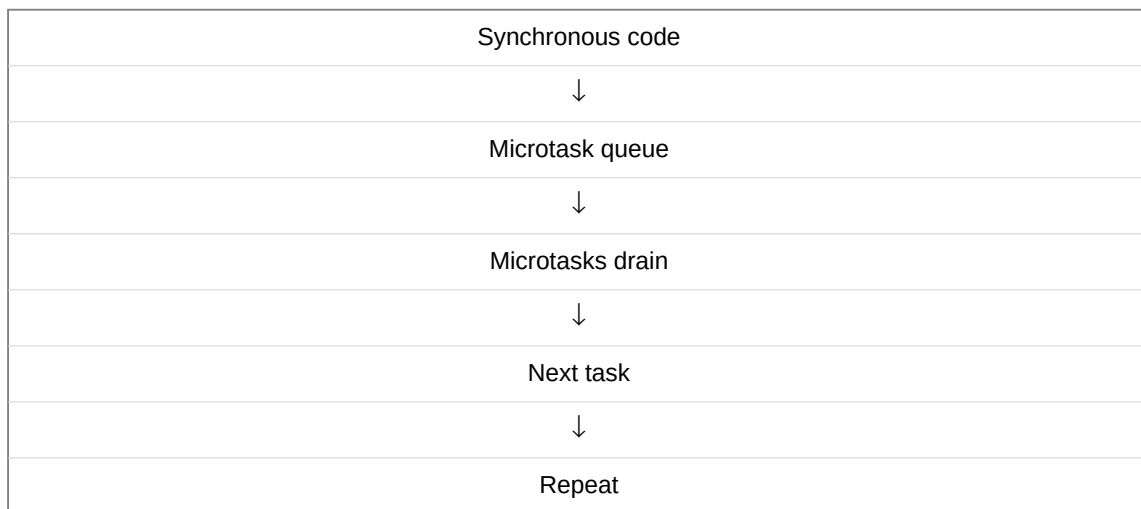
43. Microtasks vs Tasks

Definition & Theory

Promise reactions run as microtasks, while timers are tasks. Microtasks are normally processed before the next task.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
console.log("A");

setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));

console.log("D");

// A D C B
```

Conclusion

This ordering explains many common asynchronous JavaScript interview questions.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

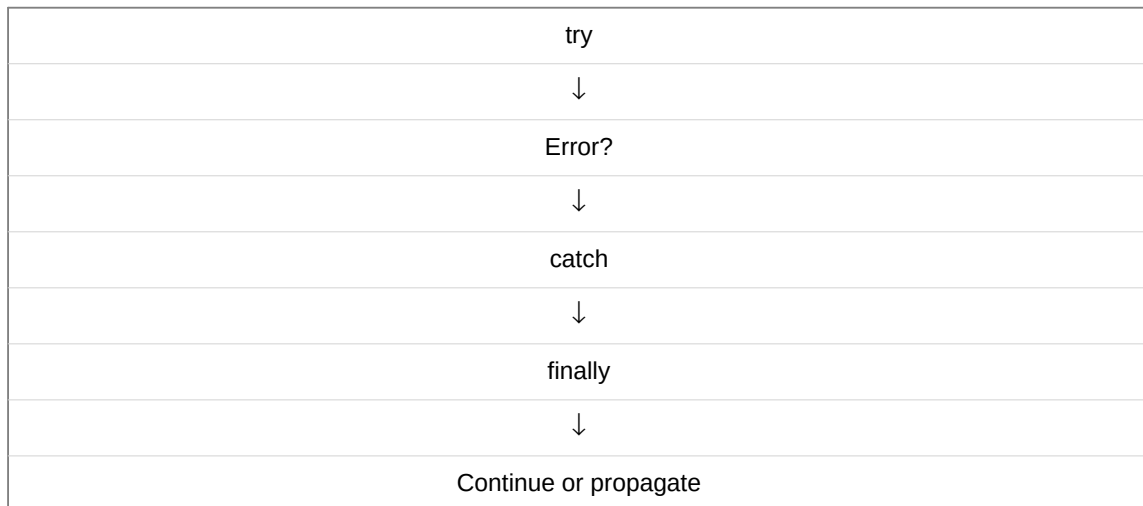
44. Error Handling

Definition & Theory

try/catch/finally provides structured handling for exceptions.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
try {  
  JSON.parse("invalid");  
} catch (error) {  
  console.error(error.message);  
} finally {  
  console.log("finished");  
}
```

Conclusion

Handle errors at a level where you can recover, report, or add useful context.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

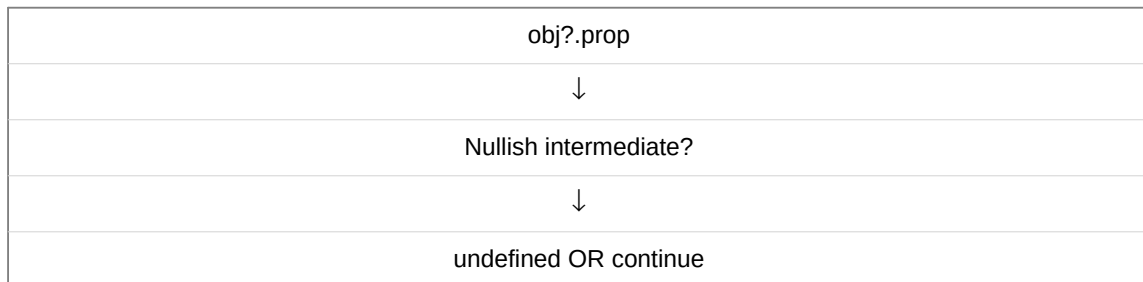
45. Optional Chaining

Definition & Theory

Optional chaining safely accesses a property when an earlier value may be null or undefined.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = {  
  profile: { city: "Hyderabad" }  
};  
  
console.log(user.profile?.city);  
console.log(user.account?.email);
```

Conclusion

It prevents a TypeError when an optional intermediate property is absent.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

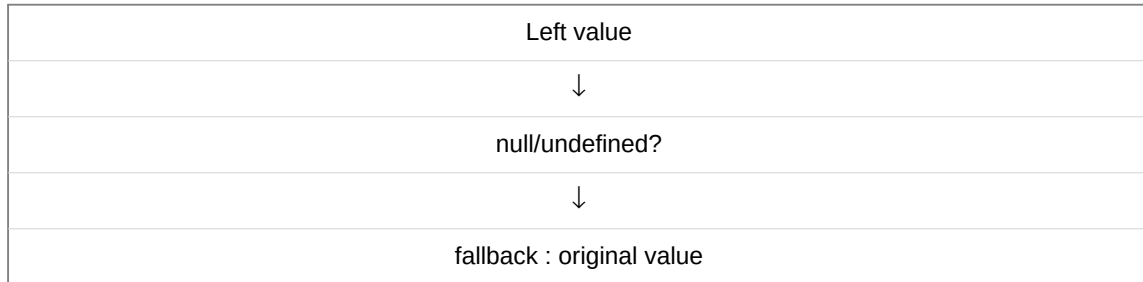
46. Nullish Coalescing

Definition & Theory

The ?? operator uses a fallback only when the left side is null or undefined.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const username = null;
console.log(username ?? "Guest");

const count = 0;
console.log(count ?? 10);
```

Conclusion

Unlike ||, ?? preserves valid falsy values such as 0 and an empty string.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

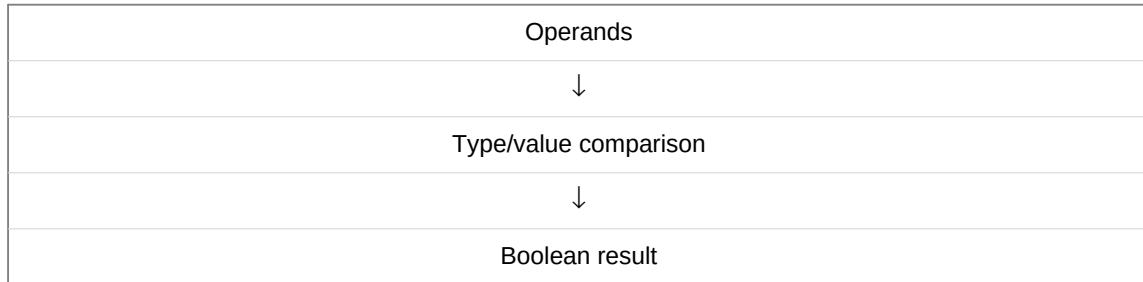
47. Equality & Coercion

Definition & Theory

Strict equality compares type and value without the implicit conversion associated with loose equality.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
console.log(5 === "5"); // false
console.log(5 == "5"); // true
console.log(null === undefined); // false
```

Conclusion

Prefer === and !== for predictable application logic.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

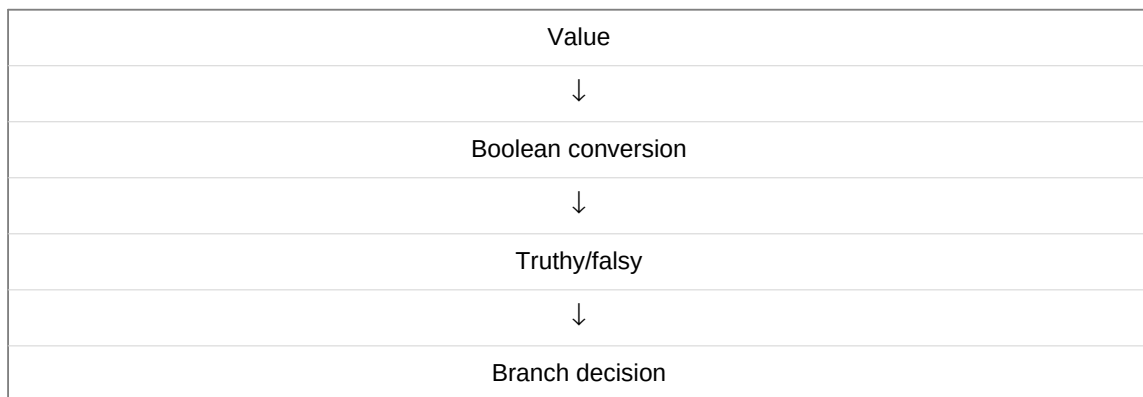
48. Truthy & Falsy

Definition & Theory

In conditional contexts JavaScript converts values to booleans. Values such as `false`, `0`, `''`, `null`, `undefined`, and `NaN` are falsy.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const values = [0, '', null, undefined, 42];

values.forEach(value => {
  console.log(Boolean(value));
});
```

Conclusion

Knowing falsy values helps explain if statements, logical operators, and defaulting patterns.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

49. Set & Map

Definition & Theory

Set stores unique values. Map stores key-value pairs and supports keys of any type.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const ids = new Set([1, 2, 2, 3]);
console.log([...ids]);

const scores = new Map();
scores.set("Sai", 95);
console.log(scores.get("Sai"));
```

Conclusion

Use Set for uniqueness and Map when you need explicit key-value storage.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.

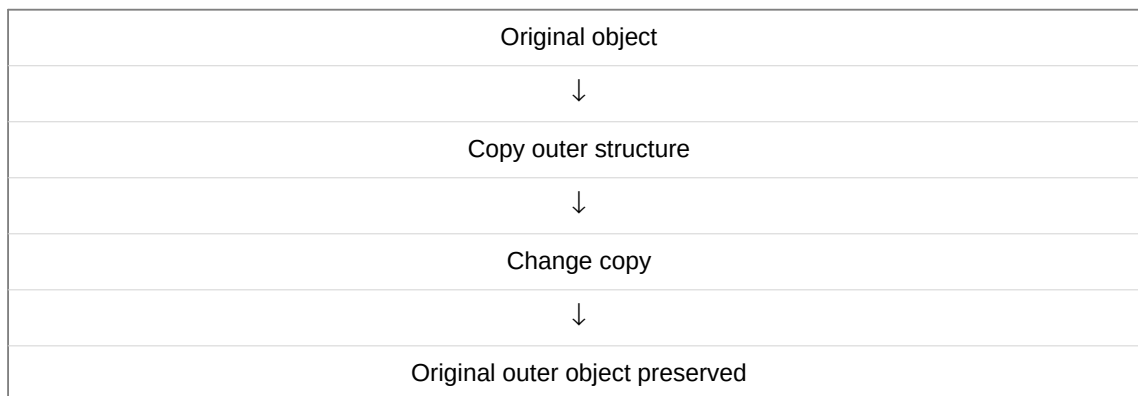
50. Immutability & Copying

Definition & Theory

Immutability means avoiding unintended changes to existing data. Spread creates a shallow copy.

The important part is to connect the syntax you type with execution. The JavaScript engine parses and executes the program, while the surrounding runtime can provide APIs and services used by the code.

How It Works — Flowchart



Code Example

```
const user = { name: "Sai", age: 20 };

const updated = {
  ...user,
  age: 21
};

console.log(user.age);
console.log(updated.age);
```

Conclusion

Spread is shallow; nested objects may still share references.

Study Check

Explain the flowchart in your own words. Then change one part of the example, predict the output, and run it to verify your reasoning.